Complete technical documentation for HydRent — **Hyderabad Rental Platform:**

🏠 HydRent — Project Documentation

1. Project Overview

HydRent is a full-stack rental property platform specialized for Hyderabad's real estate market. It connects tenants (students, working professionals, families) with property owners/agents, offering a broker-free, transparent rental experience.

Live URL: https://hydrentals.lovable.app

---

2. Tech Stack

Layer	Technology
Frontend	React 18 + TypeScript + Vite
Styling	Tailwind CSS + shadcn/ui (Radix primitives)
State Management	TanStack React Query + React Context
Routing	React Router DOM v6

Backend	Lovable Cloud (Supabase — PostgreSQL + Edge Functions)
Auth	Supabase Auth (email/password with email verification)
Maps	Leaflet.js
Charts	Recharts
i18n	Custom LanguageContext (English, Hindi, Telugu)
Testing	Vitest + Testing Library

---

3. Architecture

```
src/
├── components/
│   ├── auth/           → ProtectedRoute (RBAC guard)
│   ├── chat/           → Chatbot, ChatWindow, ConversationList
│   ├── dashboard/      → TenantDashboard, OwnerDashboard,
AdminDashboard
│   ├── home/           → Hero, Featured, Localities,
Testimonials, CTA
│   ├── layout/         → Header, Footer, MobileBottomNav,
LanguageSwitcher
│   ├── map/            → PropertyMap (Leaflet)
│   ├── notifications/  → NotificationBell (clickable → redirect)
│   ├── property/       → PropertyCard, ImageGallery,
ReviewSection, ShareDialog
```

```
│   ├── search/          → SearchBar with filters
│   ├── seo/             → SEOHead (meta tags, JSON-LD)
│   └── ui/              → 50+ shadcn/ui components
├── hooks/
│   ├── useAuth.tsx      → AuthProvider context (user, roles, profile)
│   ├── useMessages      → Real-time messaging
│   ├── usePresence      → Online status tracking
│   ├── useRealtimeNotifications → Live notification updates
│   └── useRecentlyViewed → Recently viewed properties
├── i18n/                → Translations (en, hi, te) + LanguageContext
├── pages/               → 20+ route pages
├── types/               → Property, User, Booking types
└── data/                → Mock properties for development
```

---

4. Authentication & RBAC

Roles (stored in `user_roles` table — NOT on profiles)

Role	Access
tenant	Browse, favorite, message owners, schedule visits, write reviews
owner	All tenant features + list properties, manage listings, view analytics
admin	Full access — manage users, verify properties, handle complaints

Auth Flow

1. Sign Up → User selects role (Tenant or Owner) → Email verification required

2. Sign In → Email/password → `onAuthStateChange` listener fetches profile + roles
3. Route Protection → `<ProtectedRoute requiredRoles={['admin']}>` guards pages
4. Role Check → `has_role()` SQL security-definer function used in RLS policies

Database Trigger

`handle_new_user()` automatically creates profile, assigns role, and sends welcome notification on signup.

---

5. Database Schema (16 tables)

Table	Purpose
`profiles`	User profile data (name, phone, avatar, Aadhaar, verification)
`user_roles`	RBAC roles (tenant/owner/admin)
`properties`	Property listings with all attributes
`property_views`	Analytics — view tracking
`property_reports`	Fraud/issue reports on listings

`favorites`	User saved/bookmarked properties
`saved_searches`	Saved filter configurations
`reviews`	Property ratings and comments
`messages`	In-app messaging between users
`notifications`	System notifications (typed: property_approved, new_message, etc.)
`visit_requests`	Scheduled property visit bookings
`payments`	Payment records (pending/completed/failed/refunded)
`broker_complaints`	Anti-broker complaint submissions
`community_badges`	Gamification badges (e.g., "Community Guardian 🌓")
`profiles_public`	Public view of profiles (no sensitive data)

`profiles_admin_verification`	Admin view with masked Aadhaar

---

6. Row-Level Security (RLS)

Every table has RLS enabled with granular policies:

- Properties: Anyone can view approved; owners CRUD own; admins full access
- Messages: Sender/receiver can read; sender can insert; receiver can mark read
- Notifications: Users see/update own only
- Favorites/Saved Searches: Users manage own only
- Reviews: Public read; authenticated users create; users edit/delete own
- Visit Requests: Tenants create/delete; owners/tenants update status
- Admin tables: `has_role(auth.uid(), 'admin')` check

---

7. Edge Functions

Function	Purpose	Auth
`chat`	AI chatbot for property queries	Public (no JWT)
`notify-message`	Push notification on new message	Public
`send-email`	Email delivery service	Public

---

8. Key Features

For Tenants

- 🔍 Advanced Search — Filter by locality, budget, property type, room type, furnished status, gender preference, metro proximity, pets
- 🗺️ Map View — Leaflet-based interactive map with property pins
- ❤️ Favorites — Save and compare properties
- 📊 Compare — Side-by-side property comparison
- 💬 In-App Chat — Direct messaging with property owners
- 📅 Visit Scheduling — Book property visits with date/time
- ⭐ Reviews — Rate and review properties
- 🔔 Notifications — Clickable notifications with type-based routing
- 🤖 AI Chatbot — Property discovery assistant
- 💰 Rent Calculator — Budget planning tool

For Owners

- 📝 List Properties — Multi-step form with camera capture, map pin, amenities
- 📈 Analytics Dashboard — View counts, engagement metrics (Recharts)
- 🏠 My Properties — Manage listings, update availability
- 📅 Visit Requests — Approve/reject tenant visit requests

For Admins

- ☑️ Property Verification — Approve/reject listings
- 👥 User Management — View and manage all users
- 🚨 Complaints — Handle broker complaints and property reports
- ⚙️ Settings — Platform configuration

Platform-Wide

- 🌐 Multi-language — English, Hindi, Telugu
- 📱 Mobile-First — Bottom nav, responsive layouts
- 🔒 Security — RLS, input validation triggers, data sanitization
- 📧 Email Verification — Required before sign-in
- 🏅 Gamification — Community badges for reporting brokers

---

9. Routes

Path	Component	Access
`/`	Index (Home)	Public
`/properties`	Properties listing	Public
`/property/:id`	Property detail	Public
`/map`	Map view	Public
`/auth`	Login/Signup	Public
`/dashboard`	Role-based dashboard	Authenticated
`/add-property`	Add listing	Owner/Admin
`/my-properties`	Manage listings	Owner/Admin
`/messages`	Chat inbox	Authenticated
`/notifications`	All notifications	Authenticated
`/favorites`	Saved properties	Public

`/settings`	User settings	Public
`/compare`	Compare properties	Public
`/rent-calculator`	Budget calculator	Public
`/admin/*`	Admin pages (4)	Admin only
`/forgot-password`	Password reset request	Public
`/reset-password`	Password reset	Public
`/privacy`, `/terms`, `/rules`, `/contact`	Static pages	Public

---

10. Data Validation

Server-side validation triggers on `properties` table:

- Rent: 1 – 10,000,000
- Deposit: 0 – 50,000,000
- Title: 10–200 chars
- Description: max 5,000 chars
- Pincode: exactly 6 digits (Indian format)
- Max 20 amenities, 10 images
- Sale listings require valid sale_price (max 50 Cr)

---

11. Storage

Bucket	Visibility	Purpose
`property-images`	Public	Property listing photos

---

12. Notification System

Types: `property_approved`, `property_rejected`, `new_message`, `verification_update`, `payment_received`, `general`

Auto-triggers:

- New user signup → Welcome notification
- Property status change → Approved/rejected notification
- New message → Message notification

Click routing:

- `property_approved/rejected` → `/my-properties`
- `new_message` → `/messages`
- `verification_update` → `/settings`
- Default → `/dashboard`

---

13. Environment Variables

Variable	Purpose
`VITE_SUPABASE_URL`	Backend API URL

`VITE_SUPABASE_PUBLISHABLE_KEY`	Client-side anon key
`VITE_SUPABASE_PROJECT_ID`	Project identifier

Secrets (server-side): `SUPABASE_SERVICE_ROLE_KEY`, `LOVABLE_API_KEY`, `SUPABASE_DB_URL`

---

14. Performance & SEO

- SEO: Custom `<SEOHead>` component with meta tags, JSON-LD, canonical URLs
- Sitemap: `/sitemap.xml` + `/robots.txt`
- Lazy loading: Image optimization
- HMR: Vite hot module replacement (overlay disabled)
- Query caching: TanStack React Query for server state